Week 06 – S2 PRACTICE: SQL Integration Solution

Student's Name	Seat Sreylenn
Group	**02**

Part A — Warm-ups: Build a CRUD API with Express.js + SQLite

Exercise 1 – Express + SQLite Project Setup

Goal: Scaffold a Node.js project, install Express and better-sqlite3.

Solution (package.json dependencies):

```
{
  "dependencies": {
    "express": "^4.18.2",
    "better-sqlite3": "^9.0.0",
    "dotenv": "^16.0.3"
  }
}
```

Server Entry Point (server.js):

```
const express = require('express');
const app = express();

// Middleware to parse JSON bodies
app.use(express.json());

app.get('/', (req, res) => {
  res.send('SQL warmup API running');
});

const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server running on
http://localhost:${PORT}`);
});
```

Exercise 2 – Connect my_database.db & Create the Table

Goal: Open the SQLite file and ensure the articles table exists.

Solution (utils/database.js):

```
const Database = require('better-sqlite3');
const path = require('path');

// Initialize database file
const db = new Database(path.join(__dirname, '../my_database.db'));

// Create table if it doesn't exist
db.exec(`
  CREATE TABLE IF NOT EXISTS articles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  )
`);

// Seed one article if empty (optional but helpful for testing)
const count = db.prepare('SELECT COUNT(*) as count FROM articles').get();
if (count.count === 0) {
  db.prepare('INSERT INTO articles (title, content) VALUES (?, ?)').run(
    'First Article',
    'This is the content of the first article.'
  );
}

module.exports = db;
```

Exercise 3 – GET All Articles (Read)

Goal: Return every article as JSON.

Solution (server.js):

```
const db = require('./utils/database');

app.get('/articles', (req, res) => {
  try {
    const articles = db.prepare('SELECT * FROM articles').all();
    res.json(articles);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Exercise 4 – GET One Article by ID

Goal: Read a single row using a route param and parameterized query.

Solution:

```
app.get('/articles/:id', (req, res) => {
  const { id } = req.params;
  try {
    const article = db.prepare('SELECT * FROM articles WHERE id = ?').get(id);

    if (!article) {
      return res.status(404).json({ error: 'Article not found' });
    }

    res.json(article);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Reflection: Why use <?> placeholders?

Using ? placeholders prevents **SQL Injection** attacks. If you build strings with ${id}, a malicious user could input code like 1; DROP TABLE articles;. The placeholder treats the input as data, not as executable SQL code.

Exercise 5 – POST: Create an Article

Goal: Read JSON body, INSERT a new row, return created article.

Solution:

```
app.post('/articles', (req, res) => {
  const { title, content } = req.body;

  if (!title || !content) {
    return res.status(400).json({ error: 'Title and content are required' });
  }

  try {
    const stmt = db.prepare('INSERT INTO articles (title, content) VALUES (?, ?)');
    const info = stmt.run(title, content);

    // Fetch the newly created article with the auto-generated ID
    const newArticle = db.prepare('SELECT * FROM articles WHERE id =
?').get(info.lastInsertRowid);

    res.status(201).json(newArticle);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Exercise 6 – PUT: Update an Article

Goal: Update an existing row by ID.

Solution:

```
app.put('/articles/:id', (req, res) => {
  const { id } = req.params;
  const { title, content } = req.body;

  try {
    // Check if article exists first
    const existing = db.prepare('SELECT * FROM articles WHERE id = ?').get(id);
    if (!existing) {
      return res.status(404).json({ error: 'Article not found' });
    }

    const stmt = db.prepare('UPDATE articles SET title = ?, content = ? WHERE id =
?');
    stmt.run(title, content, id);

    const updated = db.prepare('SELECT * FROM articles WHERE id = ?').get(id);
    res.json(updated);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Warning: Always include WHERE id = ?. Without it, the query updates **every** row in the table.

Exercise 7 – DELETE an Article

Goal: Remove a row and return correct status codes.

Solution:

```
app.delete('/articles/:id', (req, res) => {
  const { id } = req.params;

  try {
    const stmt = db.prepare('DELETE FROM articles WHERE id = ?');
    const result = stmt.run(id);

    if (result.changes === 0) {
      return res.status(404).json({ error: 'Article not found' });
    }

    res.status(204).send(); // No content
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Exercise 8 – Refactor into a Repository

Goal: Move SQL out of server.js into a repository module.

Repository Structure (repositories/articleRepository.js):

```
const db = require('../utils/database');

const articleRepository = {
  getAll: () => {
    return db.prepare('SELECT * FROM articles').all();
  },
  getById: (id) => {
    return db.prepare('SELECT * FROM articles WHERE id = ?').get(id);
  },
  create: (title, content) => {
    const stmt = db.prepare('INSERT INTO articles (title, content) VALUES (?, ?)');
    const info = stmt.run(title, content);
    return db.prepare('SELECT * FROM articles WHERE id =
?').get(info.lastInsertRowid);
  },
  update: (id, title, content) => {
    db.prepare('UPDATE articles SET title = ?, content = ? WHERE id =
?').run(title, content, id);
    return db.prepare('SELECT * FROM articles WHERE id = ?').get(id);
  },
  delete: (id) => {
    const stmt = db.prepare('DELETE FROM articles WHERE id = ?');
    return stmt.run(id);
  }
};

module.exports = articleRepository;
```

Updated server.js:

```
const articleRepo = require('./repositories/articleRepository');

// Example: GET All
app.get('/articles', (req, res) => {
  const articles = articleRepo.getAll();
  res.json(articles);
});
// ... repeat for other endpoints
```

Part B — Core: SQL Integration with MySQL

Exercise 9 – MySQL Manipulation

Q1 – Create the database and the articles table

Operation	SQLite Syntax	MySQL Syntax
Create	INSERT INTO ... VALUES (?, ?)	INSERT INTO ... VALUES (?, ?)
Read All	SELECT * FROM articles	SELECT * FROM articles
Read One	SELECT * FROM articles WHERE id = ?	SELECT * FROM articles WHERE id = ?
Update	UPDATE articles SET ... WHERE id = ?	UPDATE articles SET ... WHERE id = ?
Delete	DELETE FROM articles WHERE id = ?	DELETE FROM articles WHERE id = ?

```
CREATE DATABASE article_db;
USE article_db;

CREATE TABLE articles (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Q2 – Review MySQL queries

Exercise 10 – MySQL on the Backend

Layer	Responsibility
Routes	Define API endpoints (URLs) and HTTP methods (GET, POST, etc.). They receive requests and pass data to controllers.
Controllers	Handle business logic. They parse request data, call the repository, and format the response (status codes, JSON).
Repository	Handles direct database interaction. It contains all SQL queries and abstracts the database driver from the rest of the app.

(Responsibilities)

Q4 – Why 3 layers?

Q5 – Implement the database connection

```
.env
DB_HOST=localhost
DB_USER=root
DB_PORT=8889
DB_PASSWORD=root
DB_NAME=week6Db
PORT=4000
```

2. utils/database.js (Connection Pool):

```
import mysql from 'mysql2/promise';
import 'dotenv/config';

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  port: process.env.DB_PORT,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

export default pool;
```

3. repository/sqlArticleRepository.js (Async Implementation):

```
const pool = require('../utils/database');

const articleRepository = {
  getAll: async () => {
    const [rows] = await pool.query('SELECT * FROM articles');
    return rows;
  },
  getById: async (id) => {
    const [rows] = await pool.query('SELECT * FROM articles WHERE id = ?', [id]);
    return rows || null;
  },
  create: async (title, content) => {
    const [result] = await pool.query(
      'INSERT INTO articles (title, content) VALUES (?, ?)',
      [title, content]
    );
    const [newRow] = await pool.query('SELECT * FROM articles WHERE id = ?',
[result.insertId]);
    return newRow;
  },
  update: async (id, title, content) => {
    await pool.query('UPDATE articles SET title = ?, content = ? WHERE id = ?',
[title, content, id]);
    const [rows] = await pool.query('SELECT * FROM articles WHERE id = ?', [id]);
    return rows;
  },
  delete: async (id) => {
    const [result] = await pool.query('DELETE FROM articles WHERE id = ?', [id]);
    return result.affectedRows > 0;
  }
};

module.exports = articleRepository;
```

Exercise 11 – Handle Journalists

Database Update:

```
CREATE TABLE journalists (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  bio TEXT
);

-- Add foreign key to articles
ALTER TABLE articles
ADD COLUMN journalist_id INT,
ADD FOREIGN KEY (journalist_id) REFERENCES journalists(id);
```

Back End Changes:

1. Create journalistRepository similar to articleRepository.
2. Update articleRepository to join journalists:

```
getAllWithJournalist: async () => {
  const [rows] = await pool.query(`
    SELECT articles.*, journalists.name as author_name
    FROM articles
    JOIN journalists ON articles.journalist_id = journalists.id
  `);
  return rows;
}
```

Front End Changes:

1. Create a JournalistList component.
2. Add a dropdown or filter to ArticleList to filter by journalist_id.

Exercise 12 – Handle Tags (Bonus)

Database Update (Many-to-Many):

```
CREATE TABLE tags (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE article_tags (
  article_id INT,
  tag_id INT,
  PRIMARY KEY (article_id, tag_id),
  FOREIGN KEY (article_id) REFERENCES articles(id),
  FOREIGN KEY (tag_id) REFERENCES tags(id)
);
```

Back End:

- Implement getArticlesByTag(tagName) which joins articles, article_tags, and tags.

Front End:

- Add a tag selector (checkboxes or dropdown) to filter the article list.

Reflective Questions Summary

1. How does a SQL JOIN help you combine articles with journalists or categories?

A SQL JOIN allows you to retrieve related data from multiple tables in a single database query. Instead of making separate requests to the database and stitching the results together later, a JOIN matches rows based on a common column (like journalist_id in the articles table matching the id in the journalists table).

Comparison: JOIN vs. Fetching Separately

- Using JOIN: You send one query (e.g., SELECT * FROM articles JOIN journalists ON ...). The database does the heavy lifting of matching and merging. This is faster (fewer network round-trips to the DB) and ensures data consistency at the source.
- Fetching Separately: You would need to:
    1. Fetch all articles.
    2. Fetch all journalists.
    3. Write JavaScript code to loop through the articles and attach the matching journalist object to each one. This approach is slower (multiple queries), uses more memory (loading two full tables into JS), and increases the risk of logic errors if the matching logic in JavaScript is imperfect.

2. What kind of relationship exists between articles and categories, and how does it differ from the articles–journalist relationship?

Answer: The relationships differ in cardinality, which dictates how the database schema is designed:

- **Articles – Journalist (One-to-Many):**
    - **Relationship:** One journalist can write many articles, but **one article is written by only one journalist**.
    - **Schema:** This is implemented by adding a journalist_id column (Foreign Key) directly into the articles table.
- **Articles – Categories (Many-to-Many):**
    - **Relationship:** One article can have multiple tags/categories, and one category can apply to many articles.
    - **Schema:** You cannot store this in just one of the existing tables. You need a **junction table** (e.g., article_tags or article_categories) that contains two foreign keys: article_id and category_id. This allows multiple rows for the same article with different category IDs.

3. Why is the repository layer a good place to keep all SQL queries?

Answer:

The repository layer acts as a single source of truth for data access logic, enforcing the Separation of Concerns principle.

- Maintainability: If you need to change a query (e.g., adding a new column or changing a filter), you only update it in one file (sqlArticleRepository.js) rather than searching through every controller or route handler.
- Testability: You can unit test your SQL logic independently from your API logic.
- Consistency: It ensures that all data access follows the same patterns (like error handling and parameter usage).

Warm-up vs. MySQL Repository: As observed in Exercises 8 and 10, the shape of the repository remains identical regardless of the database engine.

- SQLite: Uses better-sqlite3 (synchronous or specific driver methods).
- MySQL: Uses mysql2 with pool.query (asynchronous). The *logic* (SELECT, INSERT, WHERE clauses) stays the same; only the driver method changes. This proves that the repository layer successfully abstracts the database implementation details from the rest of the application.

4. How do parameterised queries (? placeholders) protect your app, and where did you use them?

Answer: Protection Mechanism: Parameterised queries prevent **SQL Injection** attacks.

- **Without them:** If you build a query string like SELECT * FROM users WHERE id = ${userInput}, a malicious user could input 1 OR 1=1 to bypass authentication or delete data.
- **With them:** The database treats the input strictly as **data**, not as executable code.
  The ? placeholder ensures that even if a user submits malicious SQL syntax, it is escaped and treated as a literal string value.

Where they were used: You used them in **every** CRUD operation in both the SQLite and MySQL exercises:

- **GET by ID:** SELECT * FROM articles WHERE id = ?
- **POST:** INSERT INTO articles (title, content, ...) VALUES (?, ?, ...)
- **PUT:** UPDATE articles SET title = ? WHERE id = ?
- **DELETE:** DELETE FROM articles WHERE id = ?

By consistently using ? and passing the values as an array (e.g., [id] or [title, content]), you ensured the application remains secure against injection attempts.